Milestone 3: A small task list

JavaScript skeleton to add and delete tasks and save them between refreshes.

PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Milestone 3: Task List</title>
  <style>
    :root {
      --primary-color: #4f46e5;
      --primary-hover: #4338ca;
      --bg-color: #f9fafb;
      --card-bg: #ffffff;
      --text-color: #1f2937;
      --border-color: #e5e7eb;
      --danger-color: #ef4444;
      --danger-hover: #dc2626;
    }

    body {
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-color);
```

```
    margin: 0;
    padding: 2rem;
    display: flex;
    justify-content: center;
    align-items: flex-start;
    min-height: 100vh;
}

.container {
    background-color: var(--card-bg);
    padding: 2rem;
    border-radius: 12px;
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06);
    width: 100%;
    max-width: 450px;
}

h1 {
    margin-top: 0;
    font-size: 1.5rem;
    font-weight: 700;
    text-align: center;
    margin-bottom: 1.5rem;
}

.task-form {
    display: flex;
    gap: 0.5rem;
```

```
    margin-bottom: 1.5rem;
}

input[type="text"] {
    flex: 1;
    padding: 0.75rem 1rem;
    border: 1px solid var(--border-color);
    border-radius: 8px;
    font-size: 1rem;
    outline: none;
    transition: border-color 0.2s;
}

input[type="text"]:focus {
    border-color: var(--primary-color);
    box-shadow: 0 0 0 3px rgba(79, 70, 229, 0.1);
}

button {
    background-color: var(--primary-color);
    color: white;
    border: none;
    padding: 0.75rem 1.25rem;
    border-radius: 8px;
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: background-color 0.2s;
```

```
}

button:hover {
  background-color: var(--primary-hover);
}

.task-list {
  list-style: none;
  padding: 0;
  margin: 0;
}

.task-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.75rem 1rem;
  border: 1px solid var(--border-color);
  border-radius: 8px;
  margin-bottom: 0.5rem;
  background-color: #fff;
  transition: background-color 0.2s;
}

.task-item:hover {
  background-color: #f9fafb;
}
```

```
      .task-text {
        word-break: break-word;
        margin-right: 1rem;
      }

      .delete-btn {
        background-color: var(--danger-color);
        padding: 0.4rem 0.75rem;
        font-size: 0.875rem;
      }

      .delete-btn:hover {
        background-color: var(--danger-hover);
      }

      .empty-state {
        text-align: center;
        color: #9ca3af;
        font-style: italic;
        margin-top: 1rem;
      }
    </style>
</head>
<body>

    <div class="container">
      <h1>My Task List</h1>
```

```
    <!-- Task Form -->
    <form id="task-form" class="task-form">
      <input type="text" id="task-input" placeholder="Add a new task..." required autocomplete="off">
      <button type="submit" id="add-btn">Add</button>
    </form>

    <!-- Task List Container -->
    <ul id="task-list" class="task-list"></ul>
    <div id="empty-state" class="empty-state" style="display: none;">No tasks yet. Add one above!</div>
  </div>

  <script>
    // Select DOM Elements
    const taskForm = document.getElementById('task-form');
    const taskInput = document.getElementById('task-input');
    const taskList = document.getElementById('task-list');
    const emptyState = document.getElementById('empty-state');

    // Load tasks from localStorage or initialize empty array
    let tasks = JSON.parse(localStorage.getItem('tasks')) || [];

    // Initial render on page load
    renderTasks();

    // Event Listener for adding a task
    taskForm.addEventListener('submit', function (e) {
      e.preventDefault();
```

```
    const taskText = taskInput.value.trim();

    if (taskText !== '') {
        const newTask = {
            id: Date.now(), // Unique ID using timestamp
            text: taskText
        };

        tasks.push(newTask);
        saveAndRender();
        taskInput.value = ''; // Reset input field
    }
});

// Function to render tasks to the DOM
function renderTasks() {
    taskList.innerHTML = ''; // Clear current list

    if (tasks.length === 0) {
        emptyState.style.display = 'block';
        return;
    } else {
        emptyState.style.display = 'none';
    }

    tasks.forEach((task) => {
        const li = document.createElement('li');
        li.className = 'task-item';
```

```
        const span = document.createElement('span');
        span.className = 'task-text';
        span.textContent = task.text;

        const deleteBtn = document.createElement('button');
        deleteBtn.className = 'delete-btn';
        deleteBtn.textContent = 'Delete';

        // Event listener to delete specific task by ID
        deleteBtn.addEventListener('click', function () {
            deleteTask(task.id);
        });

        li.appendChild(span);
        li.appendChild(deleteBtn);
        taskList.appendChild(li);
    });
}

// Function to delete a task
function deleteTask(id) {
    tasks = tasks.filter(task => task.id !== id);
    saveAndRender();
}

// Helper function to save state and update UI
function saveAndRender() {
```

```
        localStorage.setItem('tasks', JSON.stringify(tasks));
        renderTasks();
      }
    </script>
</body>
</html>
```